

Comparative Analysis of Machine Learning and Deep Learning Models for Product Review Analysis

Elif Serra UÇAR, Nisa Begüm BURUCU, Utku ÖZTEKİN,
Deniz Kenan EK, Kerem DÖLEKSÖZ

Computer Engineering Department
Ankara Medipol University
Ankara, Turkey

elif.ucar@std.ankaramedipol.edu.tr
nisa.burucu@std.ankaramedipol.edu.tr
utku.oztekin@std.ankaramedipol.edu.tr
deniz.ek@std.ankaramedipol.edu.tr
kerem.doleksoz@std.ankaramedipol.edu.tr

Abstract. Online product reviews have become one of the most important factors affecting customer purchasing decisions in e-commerce platforms. However, the increasing number of misleading and deceptive reviews creates serious reliability problems for both customers and businesses. Because of this issue, automatic review classification and fake review detection have become important research topics in Natural Language Processing (NLP) and machine learning. In this project, we performed a comparative analysis of classical machine learning and deep learning methods for product review classification tasks using both English and Turkish datasets. The English dataset was mainly used for fake review detection, while the Turkish dataset was used for sentiment-oriented review classification due to the lack of publicly available Turkish fake review datasets. During the preprocessing stage, several NLP techniques such as lowercase conversion, punctuation removal, stopword elimination, tokenization, and sequence padding were applied. For classical machine learning approaches, Logistic Regression and Naive Bayes models were implemented using TF-IDF feature extraction. For deep learning approaches, LSTM, GRU, and BiLSTM architectures were trained using word embeddings and sequence-based representations. Experimental results showed that deep learning models achieved better performance than classical machine learning methods because they were able to capture contextual and sequential information more effectively. Among all tested models, BiLSTM achieved the highest performance on the English dataset. The Turkish dataset experiments also demonstrated strong performance of sequential deep learning architectures in multilingual review analysis tasks. Overall, this project provided practical experience in NLP preprocessing, text classification, sequence modeling, and deep learning-based review analysis systems.

Keywords. Fake Review Detection, Natural Language Processing, Deep Learning, LSTM, BiLSTM, GRU, TF-IDF, Text Classification.

1. Introduction

In recent years, online shopping platforms have become one of the most commonly used systems for purchasing products and services. Websites such as Amazon, Trendyol, Hepsiburada, eBay, and AliExpress contain millions of customer reviews that directly affect purchasing decisions. Before buying a product, most customers check user reviews in order to understand product quality, reliability, and user satisfaction. Because of this, online reviews have become extremely important for both consumers and sellers.

However, not all online reviews are trustworthy. Many reviews are intentionally written to manipulate customer opinions. Some companies or sellers may generate fake positive reviews to increase sales, while competitors may create fake negative reviews to damage a product's reputation. In addition, some reviews may be automatically generated using bots or copied from other sources. These deceptive reviews reduce user trust and create major reliability problems in e-commerce systems. Due to the increasing amount of online textual data, automatic review analysis has become an important research area in Natural Language Processing (NLP), machine learning, and deep learning. Detecting fake or suspicious reviews is a difficult problem because fake reviews are usually designed to appear realistic and human-written.

In many cases, fake reviews contain exaggerated emotional expressions, repetitive sentence patterns, unusual sentiment behavior, or suspicious writing structures. Traditional machine learning methods such as Logistic Regression and Naive Bayes have been widely used for text classification tasks because they are relatively simple, computationally efficient, and effective for sparse text representations like TF-IDF. However, these methods mainly focus on word frequency information and usually cannot fully capture sequential and contextual relationships between words. In recent years, deep learning architectures such as LSTM, GRU, and BiLSTM have become very popular in NLP tasks because they are capable of learning sequential dependencies and contextual information from text data. Unlike traditional methods, recurrent neural network-based models can better understand word order, sentence structure, and semantic relationships.

The main purpose of this project is to compare classical machine learning approaches with deep learning architectures for product review classification tasks. In this study, both English and Turkish datasets were used. The English dataset focused mainly on fake review detection, while the Turkish dataset was used for sentiment-oriented review classification due to the lack of publicly available Turkish fake review datasets. Throughout the project, several NLP preprocessing techniques were applied, including lowercase conversion, punctuation removal, stopword elimination, tokenization, TF-IDF vectorization, and sequence padding. After preprocessing, multiple machine learning and deep learning models were implemented and evaluated. The primary contributions of this project can be summarized as follows:

- Comparing classical machine learning and deep learning models on review classification tasks
- Applying NLP preprocessing techniques on multilingual review datasets
- Evaluating the effectiveness of sequential neural network architectures
- Analyzing the differ-

ences between English and Turkish review datasets\n- Investigating how contextual learning affects classification performance.

Overall, this project aims to provide both theoretical and practical understanding of modern NLP-based review analysis systems and demonstrate the advantages of deep learning methods in text classification problems.

2. Literature Review

With the rapid growth of online shopping platforms, automatic review analysis and fake review detection have become important research topics in Natural Language Processing (NLP), machine learning, and data mining. Researchers have proposed many different approaches for detecting deceptive reviews, sentiment classification, and text-based behavioral analysis.

Early studies on review classification mainly focused on traditional machine learning methods such as Logistic Regression, Naive Bayes, Support Vector Machines (SVM), and Decision Trees. These models usually relied on manually engineered text features such as bag-of-words representations, word frequencies, TF-IDF vectors, sentiment scores, punctuation usage, and review metadata.

Among classical machine learning methods, Naive Bayes became especially popular in text classification tasks because of its simplicity and efficiency. Since text datasets are usually sparse and high-dimensional, Naive Bayes can produce strong results with relatively low computational cost. Logistic Regression has also been widely used because it provides interpretable probabilistic outputs and performs well with TF-IDF representations.

One of the most commonly used feature extraction methods in NLP is TF-IDF (Term Frequency – Inverse Document Frequency). TF-IDF measures how important a word is within a document relative to the entire corpus. Unlike simple word frequency counting, TF-IDF reduces the importance of very common words while emphasizing informative terms. Because of this, TF-IDF has been widely used in sentiment analysis, spam detection, and review classification problems. Although traditional machine learning methods are computationally efficient, they have several limitations. Most classical models rely heavily on frequency-based representations and usually ignore word order and contextual meaning. For example, phrases such as: not good and good may appear similar in TF-IDF representations even though their meanings are completely different. Due to these limitations, deep learning architectures have become increasingly popular in NLP tasks. Recurrent Neural Networks (RNNs), especially LSTM and GRU models, were developed to capture sequential dependencies and contextual information from text data.

LSTM (Long Short-Term Memory) networks were introduced to solve the vanishing gradient problem of traditional RNNs. LSTM models contain memory cells and gating mechanisms that allow them to preserve long-term contextual information. Because of this capability, LSTM models have achieved strong performance in many

NLP tasks including sentiment analysis, machine translation, text generation, and fake review detection.

GRU (Gated Recurrent Unit) is a simplified version of LSTM that contains fewer parameters and requires lower computational cost. Although GRU models are computationally lighter, they can still achieve competitive performance in many sequence learning tasks.

Another important architecture used in NLP is BiLSTM (Bidirectional LSTM). Unlike standard LSTM models, BiLSTM processes text in both forward and backward directions simultaneously. This allows the model to capture richer contextual relationships from both previous and future words in a sentence. Because of this advantage, BiLSTM models often outperform standard LSTM models in text classification tasks.

Recent studies have shown that deep learning methods generally outperform traditional machine learning approaches in text-based classification problems because they can better understand semantic meaning, contextual dependencies, and sequential structures.

In multilingual NLP studies, researchers have also investigated how deep learning architectures behave across different languages. However, one major challenge in multilingual fake review detection research is the lack of high-quality labeled datasets, especially for low-resource languages such as Turkish.

In this project, we combined both classical machine learning and deep learning approaches in order to compare their effectiveness on English and Turkish review datasets. Unlike many previous studies that focus on only one language or one type of model, our project provides a comparative analysis across different datasets, preprocessing strategies, and neural network architectures.

Overall, the literature strongly suggests that sequential deep learning architectures such as LSTM and BiLSTM provide significant advantages over traditional machine learning methods in understanding contextual information within review texts.

3. Dataset Description

In this project, two different datasets were used in order to analyze product reviews using machine learning and deep learning techniques. Since the main goal of the project was to compare different NLP-based classification approaches, both English and Turkish review datasets were included in the experiments. The datasets were selected to evaluate how different models behave across different languages and review structures. One of the datasets focused on fake review detection, while the other dataset focused on sentiment-oriented review classification.

3.1 English Review Dataset

The first dataset used in this project was an English product review dataset containing approximately 5000 reviews and several metadata features related to user behavior and review characteristics.

The dataset included the following information:

Feature	Description
review_text	Main review content
rating	Product rating given by the user
sentiment_score	Sentiment polarity score
sentiment_mismatch	Difference between review sentiment and rating
verified_purchase	Indicates whether the purchase is verified
exclamation_count	Number of exclamation marks
caps_ratio	Ratio of uppercase letters
text_similarity_score	Similarity with other reviews

Unlike standard sentiment analysis datasets, this dataset contained behavioral and stylistic features that could help identify suspicious review patterns.

One of the most important steps in this dataset was the label generation process. Since the dataset did not directly contain a fake review label, labels were generated using the sentiment mismatch feature.

The following rule was used in our project:

```
df["label"] = (df["sentiment_mismatch"] > 0.2).astype(int)
```

Using this rule:

- reviews with high sentiment mismatch were labeled as suspicious/fake
- reviews with lower mismatch values were labeled as normal/genuine

This created a binary classification problem:

Label 0 means genuine review, label 1 means fake review.

The main purpose of using this dataset was to investigate whether machine learning and deep learning models could detect suspicious review behavior using textual and behavioral information together.

3.2 Turkish Review Dataset

The second dataset used in this project was a Turkish product review dataset.

One of the challenges during the project was the lack of publicly available Turkish fake review datasets.

Most available Turkish datasets were designed for sentiment analysis rather than fake review detection. Because of this limitation, a Turkish review dataset containing product ratings and customer comments was selected.

The dataset mainly included:

Feature review means user review text, rating(star) means product rating. Since the dataset did not contain fake review labels, a sentiment-oriented binary classification approach was applied.

The following labeling strategy was used:

1 if rating ≥ 4 else 0

In this setup:

- ratings of 4 and 5 were considered positive reviews
- ratings of 1 and 2 were considered negative reviews

Additionally, 3-star reviews were removed because they represent neutral opinions and could make the classification problem less clear.

This preprocessing step was implemented as follows:

```
df_tr = df_tr[df_tr["Rating (Star)"] != 3]
```

After preprocessing, the Turkish dataset was used mainly to analyze how sequential deep learning models behave on Turkish review texts.

Although the Turkish dataset was not originally designed for fake review detection, it still provided valuable insights into multilingual review classification and NLP model performance across different languages.

3.3 Dataset Comparison

Property	English Dataset	Turkish Dataset
Language	English	Turkish
Main Task	Fake Review Detection	Sentiment Classification
Label Type	Generated using sentiment mismatch	Generated using ratings
Metadata Features	Available	Limited
Sequential Text Analysis	Yes	Yes

Using two different datasets allowed us to compare not only different machine learning approaches, but also the behavior of NLP models across different languages and classification tasks.

4. Data Preprocessing

Data preprocessing is one of the most important stages in Natural Language Processing (NLP) projects. Raw text data collected from online platforms usually contains many unnecessary elements such as punctuation marks, uppercase letters, special characters, repeated words, emojis, and stopwords. These elements may create noise and negatively affect model performance.

Because of this, several preprocessing techniques were applied before training the machine learning and deep learning models. The main purpose of preprocessing was to clean the text data, reduce noise, standardize text structure, and transform reviews into machine-readable representations.

The preprocessing pipeline used in this project is shown below:

Raw Review Text → Lowercase Conversion → Punctuation Removal → Stopword Removal → Text Cleaning → Tokenization → Sequence Padding / TF-IDF → Model Training

4.1 Lowercase Conversion

The first preprocessing step was converting all text into lowercase format.

Example:

Original Text	Processed Text
THIS PRODUCT IS AMAZING	this product is amazing

This step was implemented using:

```
text = text.lower()
```

The purpose of lowercase conversion is to reduce vocabulary size and standardize text representation. Without this step, words such as:

- Product
- PRODUCT
- product

would be treated as completely different words by the model.

Lowercase conversion helps the model focus on semantic meaning rather than formatting differences.

4.2 Removing Punctuation and Special Characters

Online reviews often contain unnecessary punctuation marks, emojis, URLs, symbols, and repeated characters.

Example:

Original Text	Cleaned Text
Amazing!!!! Best product :)	Amazing Best product

This step helps reduce text noise and improves the quality of extracted features.

Removing punctuation is especially useful in TF-IDF representations because punctuation symbols usually do not contribute meaningful semantic information. `re.sub(r"[^\w\s]", "", text)`

4.3 Stopword Removal

Stopwords are extremely common words that usually carry very little semantic meaning. Like the, is, a, an, and, of.

These words appear frequently in almost every sentence and generally do not help distinguish between classes. For stopwords removal, the NLTK stopwords library was used.

this product is very good → product good

The purpose of stopwords removal is:

- reducing dimensionality
- decreasing noise
- improving feature quality
- helping the model focus on informative words

However, stopwords removal should be applied carefully because some stopwords may affect sentence meaning in certain NLP tasks.

4.4 Text Cleaning

Additional text cleaning operations were also applied to improve overall dataset quality.

These operations included:

- removing extra whitespace
- removing repeated symbols
- removing unnecessary line breaks
- normalizing text structure

This step improves consistency across the dataset and helps the model process reviews more effectively.

4.5 Tokenization

Machine learning models cannot directly process raw text data. Therefore, review texts must be converted into numerical representations.

For deep learning models, tokenization was applied.

Tokenizer converts words into integer-based sequences.

Example:

this product amazing → [15,88,401]

Implementation:

```
tokenizer.texts_to_sequences(texts)
```

The tokenizer creates a vocabulary dictionary and assigns a unique integer ID to each word.

This step is extremely important for sequence-based deep learning architectures such as:

- LSTM
- GRU
- BiLSTM

because these models process numerical sequences instead of raw text.

4.6 Sequence Padding

One of the major challenges in NLP tasks is that review lengths vary significantly.

Example:

Review	Length
Good	1
This product is extremely useful	5

However, neural networks require fixed-length input vectors.

To solve this problem, sequence padding was applied using:

```
pad_sequences(maxlen=100)
```

This operation ensures that all review sequences have the same length.

If a review is shorter than the maximum length, padding values are added.

If a review is longer, extra words are truncated.

Padding provides several advantages:

- standardized model input
- easier batch processing
- compatibility with neural network architectures

4.7 TF-IDF Preparation for Classical Machine Learning

For classical machine learning models such as Logistic Regression and Naive

Bayes, TF-IDF vectorization was used instead of sequence representations.

```
TfidfVectorizer(max_features=5000)
```

TF-IDF transforms text into sparse numerical feature vectors.

The purpose of TF-IDF is to identify words that are important within a review while reducing the influence of extremely common words.

Example informative words: excellent,terrible,refund,fake,waste.

TF-IDF representations work particularly well with classical machine learning algorithms because they efficiently represent high-dimensional text data.

4.8 Importance of Preprocessing

Preprocessing significantly affects NLP model performance. Poor-quality preprocessing can reduce accuracy and increase noise within the dataset.

The preprocessing stage in this project helped:

- normalize review texts
- reduce irrelevant information
- improve feature quality
- simplify vocabulary
- prepare data for deep learning architectures

Overall, preprocessing created a cleaner and more structured dataset, which improved both machine learning and deep learning model performance.

5. Feature Extraction and Text Representation

After preprocessing the review texts, the next step was converting textual data into numerical representations that machine learning and deep learning models could understand.

Since machine learning algorithms cannot directly process raw text, feature extraction techniques were applied to transform review data into vectors and sequences.

In this project, two different text representation approaches were used:

1. TF-IDF representation for classical machine learning models

2. Tokenized sequence representation for deep learning models

The reason for using two different approaches is that traditional machine learning algorithms and deep learning architectures process textual data differently.

5.1 TF-IDF Representation

For classical machine learning models such as:

- Logistic Regression
- Naive Bayes

TF-IDF vectorization was used.

```
vectorizer = TfidfVectorizer(max_features=5000)
```

```
X = vectorizer.fit_transform(df["clean_text"])
```

TF-IDF stands for: Term Frequency – Inverse Document Frequency

This method measures how important a word is within a document relative to the entire dataset.

The intuition behind TF-IDF is:

- frequently occurring words inside a review are important
- words appearing in almost every review become less informative

Word	Importance
Amazing	High
Terrible	High
Product	Medium
the	Very low

Words like “the” or “and” appear everywhere, so TF-IDF gives them low importance scores.

However, more discriminative words such as: fake, refund, scam, excellent, terrible. Receive higher weights because they help distinguish review classes.

5.2 Why TF-IDF Was Used

TF-IDF is widely used in Natural Language Processing tasks because it is:

- simple
- computationally efficient
- interpretable
- effective for sparse text classification

It performs especially well with traditional machine learning algorithms.

In this project, TF-IDF was selected because:

- the dataset contains textual reviews
- word importance matters in fake review detection
- classical machine learning models require fixed numerical vectors

Another advantage of TF-IDF is dimensionality reduction through the parameter: `max_features=5000`

This limits the vocabulary size to the 5000 most important words.

Without this limitation:

- feature space becomes extremely large
- memory usage increases
- model training becomes slower
- overfitting risk increases

Therefore, limiting feature count improves efficiency while preserving important information.

5.3 Sparse Matrix Representation

TF-IDF produces sparse matrices.

- most values are zero

- only important words receive non-zero values

Word	Review 1	Review 2
Amazing	0.72	0
terrible	0	0.81

Sparse representations are memory efficient and highly compatible with:

- Logistic Regression
- Naive Bayes

This is one reason why these models are commonly used in text classification problems.

5.4 Tokenization for Deep Learning

Unlike classical machine learning models, deep learning architectures process sequential information.

Therefore, review texts were converted into sequences using tokenization.

```
tokenizer = Tokenizer(num_words=5000)
```

```
tokenizer.fit_on_texts(df["clean_text"])
```

```
X_seq = tokenizer.texts_to_sequences(df["clean_text"])
```

The tokenizer creates a vocabulary dictionary where every word receives a unique integer ID.

Word	Integer ID
amazing	25
Product	87
fake	302

A review sentence is then transformed into a sequence:

Amazing product → [25,87]

This representation preserves word order, which is extremely important for sequence-based neural networks.

5.5 Importance of Word Order

One limitation of TF-IDF is that it ignores word order.

For example: not good, good not may produce similar feature vectors even though their meanings differ.

Deep learning models such as:

- LSTM
- GRU
- BiLSTM

solve this problem by processing reviews sequentially.

This allows the models to learn:

- contextual meaning
- word dependencies
- sentence structure
- semantic relationships

As a result, deep learning architectures generally outperform traditional machine learning methods in NLP tasks.

5.6 Sequence Padding

Neural networks require equal-length input sequences.

However, review lengths vary significantly.

To solve this issue, sequence padding was applied.

`X_pad = pad_sequences(X_seq, maxlen=100)`

Padding ensures that every review contains exactly 100 tokens.

Short reviews are padded with zeros, while long reviews are truncated.

5.7 Comparison Between TF-IDF and Token Sequences

Feature	TF-IDF	Token Sequences
---------	--------	-----------------

Used In	ML models	Deep learning
Preserves word order	No	Yes
Sparse representation	Yes	No
Computational cost	Lower	Higher
Semantic learning	Limited	Strong
Context awareness	Weak	Strong

TF-IDF works efficiently for simpler machine learning models, while token sequences provide richer semantic information for neural network architectures.

Because of this, both approaches were included in the project for comparative analysis.

5.8 Importance of Feature Extraction

Feature extraction is one of the most critical stages in NLP systems.

The quality of extracted features directly affects model performance.

In our project, feature extraction helped:

- transform raw reviews into numerical data
- reduce noise
- improve semantic learning
- enable sequence learning
- improve classification performance

Using both TF-IDF and sequence-based representations allowed the project to compare traditional machine learning approaches with modern deep learning techniques in fake review detection.

6. Machine Learning Models

In our project, classical machine learning algorithms were first applied before using deep learning architectures. The purpose of this stage was to establish baseline performance results and compare traditional approaches with neural network-based models.

Two machine learning algorithms we were used:

- Logistic Regression
- Naive Bayes

These algorithms are widely used in Natural Language Processing tasks because they are computationally efficient and perform well on text classification problems.

The models were trained using TF-IDF feature representations extracted from review texts.

6.1 Logistic Regression

Logistic Regression is one of the most commonly used supervised machine learning algorithms for binary classification problems.

In this project, it was used to classify reviews into two categories:

- Fake Review
- Real Review

```
lr = LogisticRegression()
```

```
lr.fit(X_train, y_train)
```

```
y_pred = lr.predict(X_test)
```

Logistic Regression is actually a classification algorithm despite its name.

The model predicts probabilities using the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

If the probability is greater than 0.5, the review is classified as fake; otherwise, it is classified as real.

6.2 Why Logistic Regression Was Selected

Logistic Regression was selected because:

- it performs efficiently on high-dimensional sparse text data
- it works well with TF-IDF vectors
- it is fast to train
- it provides strong baseline performance
- it is easy to interpret

Since TF-IDF generates sparse matrices with thousands of features, Logistic Regression becomes a suitable choice for NLP classification tasks.

Another advantage is that the model is computationally lightweight compared to deep learning architectures.

6.3 Logistic Regression Results

The Logistic Regression model achieved the following accuracy:

Model	Accuracy
Logistic Regression	0.782

This result indicates that the model successfully learned meaningful patterns from textual reviews.

However, the model still has limitations because it does not fully capture sequential dependencies and contextual information within sentences.

For example not good, good may still produce similar vector representations under certain conditions.

This limitation motivated the use of deep learning architectures later in the project.

6.4 Naive Bayes

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' Theorem.

In this project, the Multinomial Naive Bayes version was used because it is highly effective for text classification tasks.

```
nb = MultinomialNB()
```

```
nb.fit(X_train, y_train)
```

```
y_pred_nb = nb.predict(X_test)
```

Naive Bayes calculates the probability that a review belongs to a specific class based on the words contained in the review.

The algorithm assumes that words are conditionally independent from each other.

Although this assumption is mathematically unrealistic in natural language, Naive Bayes still performs surprisingly well in many NLP applications.

$$P(c | x) = \frac{P(x | c) P(c)}{P(x)}$$

Likelihood
Class Prior Probability

Posterior Probability
Predictor Prior Probability

$$P(c | X) = P(x_1 | c) \times P(x_2 | c) \times \dots \times P(x_n | c) \times P(c)$$

6.5 Why Naive Bayes Was Selected

Naive Bayes was included in the project because:

- it is widely used in NLP
- it performs well on TF-IDF features
- it is extremely fast
- it works efficiently on sparse datasets
- it provides a strong traditional baseline

Naive Bayes is especially popular for:

- spam detection
- sentiment analysis
- email filtering
- document classification

Its computational simplicity makes it highly scalable for large text datasets.

6.6 Naive Bayes Results

The Naive Bayes model achieved the following accuracy:

Model	Accuracy
Naive Bayes	0.794

The model slightly outperformed Logistic Regression in this project.

One possible reason is that probabilistic modeling works effectively on word-frequency-based text representations such as TF-IDF.

However, Naive Bayes still suffers from several limitations:

- independence assumption between words
- inability to capture long-term context
- weak sequential understanding

Because of these limitations, deep learning models were later introduced to improve contextual learning and sequence understanding.

6.7 Comparison of Classical Machine Learning Models

Model	Advantages	Limitations
Logistic Regression	Simple, interpretable, fast	Weak contextual learning
Naive Bayes	Efficient, scalable, strong baseline	Independence assumption

Both models produced reasonable classification performance despite their simplicity.

However, their inability to capture sequential dependencies motivated the transition toward recurrent neural network architectures such as:

- LSTM
- GRU
- BiLSTM

which are discussed in the next section.

7. Deep Learning Models

After applying classical machine learning algorithms, deep learning architectures were implemented to improve contextual understanding and sequential learning from review texts.

Unlike traditional machine learning methods, deep learning models can process word sequences and capture semantic relationships between words within sentences.

In this project, three recurrent neural network architectures were used:

- LSTM
- GRU
- BiLSTM

These models were selected because fake review detection depends heavily on contextual meaning, sentence structure, and word dependencies.

Deep learning models were trained using tokenized and padded review sequences instead of TF-IDF vectors.

7.1 Why Deep Learning Was Used

Traditional machine learning models such as Logistic Regression and Naive Bayes mainly rely on word frequency statistics.

Although these approaches are computationally efficient, they have several limitations:

- inability to capture long-term dependencies
- weak contextual understanding
- limited semantic learning
- inability to preserve word order effectively

Deep learning architectures solve many of these problems by processing reviews sequentially.

This allows models to learn:

- sentence context
- word relationships
- semantic meaning
- sequential dependencies

As a result, deep learning methods generally achieve higher performance in NLP classification tasks.

7.2 Embedding Layer

All deep learning models in this project started with an Embedding layer.

```
Embedding(input_dim=5000, output_dim=64)
```

The embedding layer transforms integer-based word IDs into dense vector representations.

Instead of representing words as sparse vectors, embeddings map words into continuous semantic spaces.

Example:

Word	Embedding Vector
amazing	[0.21, -0.44, 0.91, ...]
terrible	[-0.71, 0.15, -0.22, ...]

Semantically similar words tend to appear closer within embedding space.

This improves the model's ability to understand contextual similarities between words.

7.3 LSTM Model

LSTM stands for Long Short-Term Memory

LSTM is a special type of recurrent neural network (RNN) designed to learn long-term sequential dependencies.

```
model.add(LSTM(64))
```

Traditional RNN models suffer from the vanishing gradient problem, meaning they struggle to remember earlier information in long sequences.

LSTM solves this issue using memory cells and gating mechanisms.

The architecture can selectively:

- remember important information
- forget irrelevant information
- preserve long-term dependencies

This makes LSTM highly effective for NLP tasks involving sequential text processing.

7.4 Dropout Regularization

To reduce overfitting, Dropout regularization was applied.

```
model.add(Dropout(0.5))
```

Dropout randomly disables neurons during training.

This prevents the model from memorizing training data too strongly and improves generalization performance.

Without dropout:

- training accuracy may become extremely high
- test accuracy may decrease
- overfitting risk increases

Using dropout helped the model become more robust on unseen review data.

7.5 Model Compilation

The models were compiled using:

```
loss='binary_crossentropy'
```

```
optimizer='adam'
```

```
metrics=['accuracy']
```

Binary Crossentropy

Binary crossentropy was selected because the problem is a binary classification task.

The loss function measures prediction error between actual labels and predicted probabilities.

Adam Optimizer

Adam optimizer was used because:

- it converges quickly
- it adapts learning rates automatically
- it performs efficiently on NLP tasks

Adam is one of the most commonly used optimizers in deep learning applications.

7.6 Model Training

epochs=3

batch_size=32

Epoch:

An epoch means one complete pass through the entire training dataset.

In this project:

- the dataset was processed 3 times during training

Batch Size:

Batch size determines how many samples are processed before updating model weights.

Using batch size 32 provides a balance between:

- computational efficiency
- stable gradient updates
- memory usage

7.7 GRU Model

```
gru_model.add(GRU(64))
```

GRU simplifies the LSTM architecture by using fewer gates.

Advantages of GRU include:

- faster training

- lower computational complexity
- fewer parameters
- efficient sequential learning

Despite being simpler, GRU often performs similarly to LSTM in many NLP tasks.

7.8 BiLSTM Model

Bidirectional(LSTM(64))

Unlike standard LSTM, BiLSTM processes text in both directions:

- left-to-right
- right-to-left

This allows the model to learn contextual information from both past and future words simultaneously.

7.9 Deep Learning Model Results

The deep learning models achieved the following results on the English dataset:

Model	Accuracy
LSTM	0.819
GRU	0.815
BiLSTM	0.825

BiLSTM achieved the highest accuracy among all English dataset models.

The Turkish dataset produced even higher results:

Model	Accuracy
LSTM	0.968
GRU	0.962
BiLSTM	0.967

The higher Turkish dataset performance may be related to:

- cleaner labels

- stronger sentiment polarity
- easier class separation
- dataset structure differences

8. Experimental Results

This section presents the experimental results obtained from both classical machine learning models and deep learning architectures.

The main objective of the experiments was to compare model performances for fake review detection on both English and Turkish review datasets.

Model performance was evaluated primarily using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

8.1 English Dataset Results

The English fake review dataset was evaluated using both machine learning and deep learning models.

The obtained accuracy scores are shown below.

Model	Accuracy
Logistic Regression	0.782
Naive Bayes	0.794
LSTM	0.819
GRU	0.815
BiLSTM	0.825

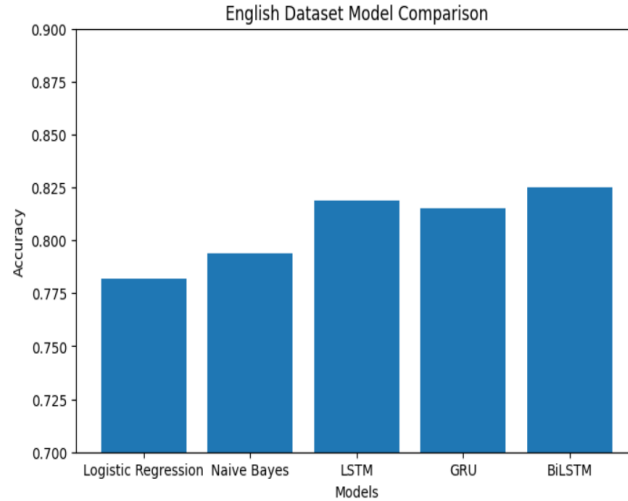


Fig. X. Accuracy comparison of machine learning and deep learning models on the English dataset.

8.2 Analysis of English Dataset Results

The experimental results show that deep learning architectures outperformed traditional machine learning methods on the English dataset.

Among all models :

BiLSTM achieved the highest accuracy with a score of 0.825.

One major reason for this improvement is BiLSTM's ability to process textual information bidirectionally.

Unlike TF-IDF-based machine learning methods, recurrent neural networks preserve sequential information and contextual dependencies between words.

Although Logistic Regression and Naive Bayes produced acceptable results, their contextual understanding capabilities remain limited because they mainly depend on word frequency statistics.

The results indicate that contextual sequence learning plays an important role in fake review detection tasks.

8.3 Turkish Dataset Results

The Turkish dataset was also evaluated using deep learning architectures.

The obtained accuracy scores are shown below.

Model	Accuracy
LSTM	0.968
GRU	0.962
BiLSTM	0.967

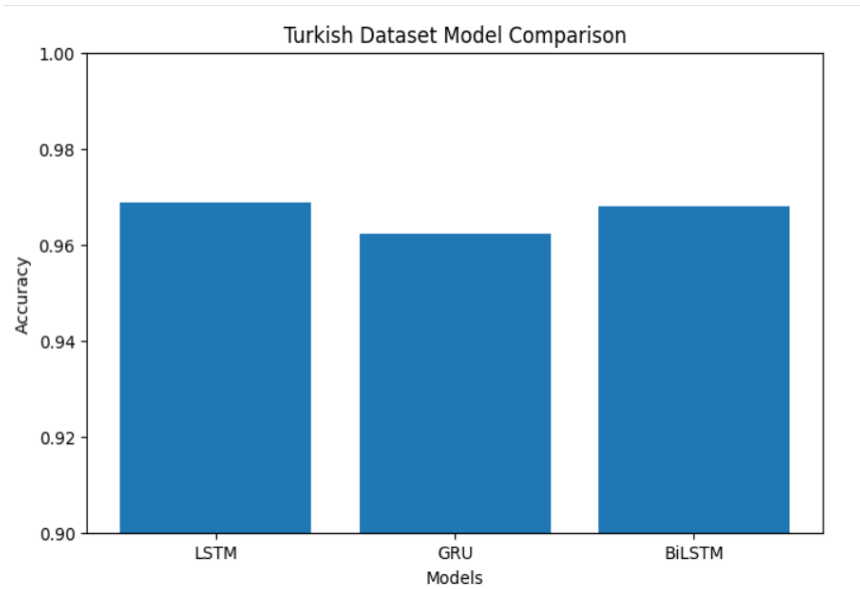


Fig. X. Accuracy comparison of deep learning models on the Turkish dataset.

8.4 Analysis of Turkish Dataset Results

The Turkish dataset produced significantly higher accuracy scores compared to the English dataset.

Several possible reasons may explain this difference:

- stronger sentiment polarity in Turkish reviews
- cleaner dataset structure
- easier class separation

- simpler binary labeling strategy

The Turkish dataset labeling process was based on rating scores:

- ratings greater than or equal to 4 were labeled as positive
- ratings below 4 were labeled as negative

This labeling strategy may have produced clearer class boundaries compared to the English fake review dataset.

Among Turkish models: LSTM achieved the highest accuracy with 0.968.

The results demonstrate that recurrent neural networks perform highly effectively on Turkish review classification tasks.

8.5 Confusion Matrix Analysis

To further evaluate model performance, a confusion matrix was generated for the English BiLSTM model.

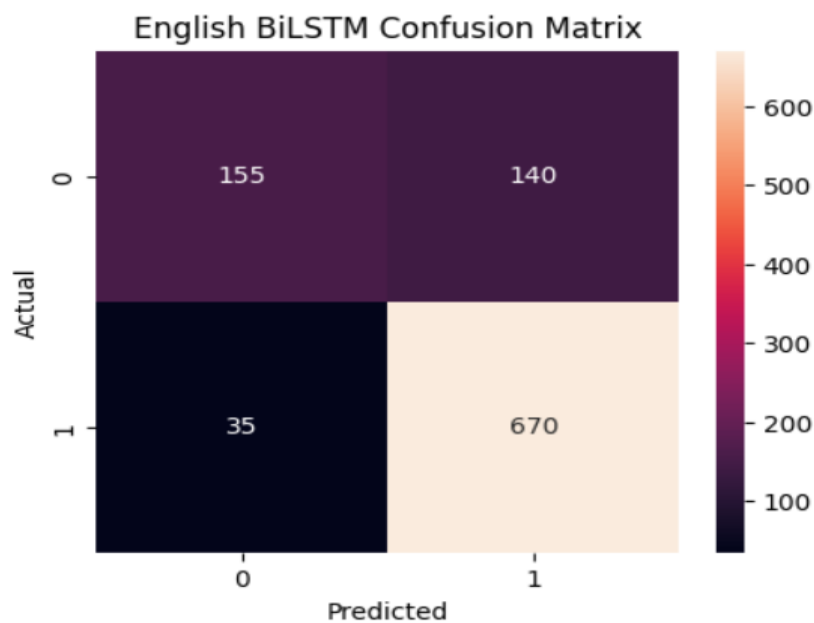


Fig. X. Confusion matrix of the English BiLSTM model.

The confusion matrix provides detailed information about prediction performance.

It shows:

- correctly classified fake reviews
- correctly classified real reviews
- false positives
- false negatives

The matrix indicates that the model successfully identified most fake reviews with relatively high precision and recall values.

However, some real reviews were still incorrectly classified as fake reviews, indicating the presence of false positive predictions.

This suggests that certain linguistic patterns within genuine reviews may resemble fake review behavior.

8.6 Classification Report Analysis

The classification report for the English BiLSTM model is presented below.

	precision	recall	f1-score	support
0	0.82	0.53	0.64	295
1	0.83	0.95	0.88	705
accuracy			0.82	1000
macro avg	0.82	0.74	0.76	1000
weighted avg	0.82	0.82	0.81	1000

The classification report evaluates the model using precision, recall, and F1-score metrics.

Precision

Precision measures how many predicted fake reviews were actually fake.

The model achieved strong precision values for both classes, indicating reliable classification capability.

Recall

Recall measures how many actual fake reviews were successfully detected.

The recall score for fake reviews was particularly high, demonstrating that the model effectively captured suspicious review patterns.

F1-score

F1-score balances precision and recall.

The obtained F1-scores indicate that the BiLSTM model achieved stable overall classification performance.

8.7 Overall Experimental Findings

The experimental results demonstrate several important findings:

- deep learning models outperformed classical machine learning algorithms
- sequence-based architectures improved contextual understanding
- BiLSTM achieved the strongest English dataset performance
- Turkish dataset classification achieved very high accuracy
- preprocessing and feature extraction significantly improved performance

Overall, the experiments indicate that recurrent neural network architectures are highly effective for fake review detection and sentiment-based text classification tasks.

9. Discussion

The results obtained in this project demonstrate that deep learning architectures are generally more effective than traditional machine learning methods for review classification tasks.

One of the main reasons for this performance difference is the ability of recurrent neural networks to learn sequential and contextual information from text data. While classical machine learning models mainly rely on word frequency statistics, deep learning models can better understand sentence structure, word order, and semantic relationships.

Among all tested models, BiLSTM achieved the highest performance on the English dataset. This indicates that bidirectional contextual learning improves fake review detection capability.

Another important observation is the significantly higher performance obtained on the Turkish dataset. One possible explanation is that the Turkish dataset focused on sentiment-oriented classification rather than direct fake review detection. Since the labels were generated using rating scores, class boundaries may have become

clearer and easier for the models to learn.

The preprocessing stage also had a major impact on model performance. Techniques such as lowercase conversion, punctuation removal, stopword elimination, tokenization, and sequence padding helped reduce noise and improve feature quality. Despite the strong results, several limitations were encountered during the project. One major challenge was the lack of publicly available Turkish fake review datasets. Because of this limitation, sentiment-based labeling had to be used for the Turkish dataset.

Another limitation is the computational cost of deep learning models. Compared to Logistic Regression and Naive Bayes, recurrent neural network architectures required more training time and computational resources.

Overall, the project demonstrated that sequential deep learning models such as LSTM and BiLSTM are highly effective for NLP-based review classification tasks and provide stronger contextual learning capabilities compared to traditional machine learning approaches.

10. Conclusion

In this project, a comparative analysis of classical machine learning and deep learning approaches was performed for product review classification tasks using English and Turkish datasets.

During the project, several NLP preprocessing techniques such as text cleaning, stopword removal, TF-IDF vectorization, tokenization, and sequence padding were applied in order to prepare the datasets for machine learning and deep learning models.

Classical machine learning algorithms including Logistic Regression and Naive Bayes produced reasonable classification performance and provided strong baseline results. However, deep learning architectures achieved better overall performance because they were able to capture contextual and sequential relationships within review texts.

Among all tested models, BiLSTM achieved the highest performance on the English dataset, demonstrating the importance of bidirectional contextual learning in fake review detection tasks.

The Turkish dataset experiments also showed that recurrent neural network architectures perform very effectively on sentiment-oriented review classification problems.

- Natural Language Processing
- Text preprocessing
- Machine learning
- Deep learning
- Sequence modeling
- Model evaluation
- Review classification systems

In future work, more advanced transformer-based architectures such as BERT and attention-based models can be explored to further improve multilingual fake review detection performance.

11. References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
2. Chollet, F. (2021). *Deep Learning with Python*. Manning Publications.
3. NLTK Documentation. <https://www.nltk.org/>
4. TensorFlow Documentation. <https://www.tensorflow.org/>
5. Scikit-learn Documentation. <https://scikit-learn.org/>
6. Keras Documentation. <https://keras.io/>
7. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, 2825–2830.
8. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O’Reilly Media.
9. <https://www.kaggle.com/datasets/sathishcrispsystem/fake-product-review?resource=download>
10. <https://www.kaggle.com/datasets/cebeci/turkishreviews>
11. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill Education.
12. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques*. 3rd Edition, Morgan Kaufmann Publishers.
13. Özzyer, T. *Data Science Lecture Notes*. Ankara Medipol University

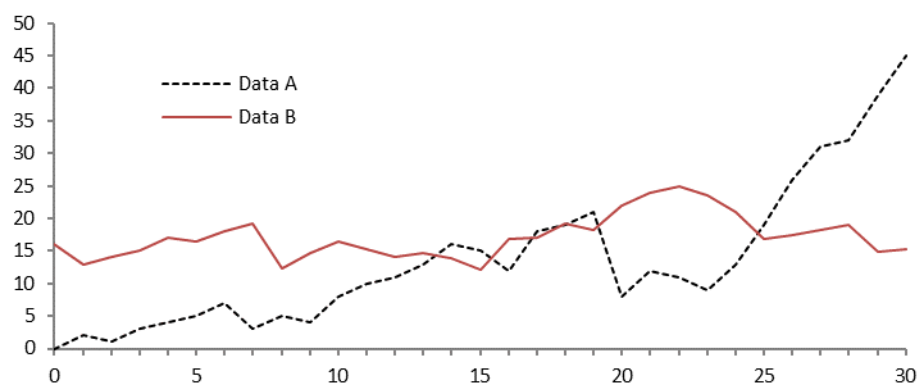


Fig. 1. A figure caption is always placed below the illustration. Short captions are centered, while long ones are justified. The macro button chooses the correct format automatically.

For citations of references, we prefer the use of square brackets and consecutive numbers. Citations using labels or the author/year convention are also acceptable. The following bibliography provides a sample reference list with entries for journal articles [1], an LNCS chapter [2], a book [3], proceedings without editors [4], as well as a URL [5].

References

1. Author F (2016) Article title. *Journal* 2(5):99–110.
2. Author F, Author S (2017) Title of a proceedings paper. In: Editor F, Editor S (eds.) *Proceedings Title*, pp. 1–13. Springer, Heidelberg.
3. Author F, Author S, Author T (1999) *Book title*. 2nd edn. Publisher, Location.
4. Author F (2010) Contribution title. In: 9th International Proceedings on Proceedings, pp. 1–2. Publisher, Location.
5. LNCS Homepage, <http://www.springer.com/lncs>, last accessed 2016/11/21.